SQL LAB EXAM · MODEL QUESTIONS

# Every Model Question, Taught

All questions from **Model-questions – final.pdf**, in order. For each one: the exact question, **how to think** (which recipe and why), the answer you type, and the **real output** it prints. Each schema runs from an empty database, top to bottom, exactly as you would in the lab.


1 · Student Management

2 · Employee Management

3 · Railway Reservation

4 · Hospital Management

5 · Bank Management

6 · Airline Reservation

7 · Movie Booking

8 · Food Delivery

## How to use this

**1.** Read the purple question.
**2.** Cover the answer and say which recipe you'd use.
**3.** Check the **How to think** box, then the answer.
**4.** Look at the output: that's what proves it worked.

Grey **Setup** boxes are not asked in the question, but you need them so the next question has tables/data. In the exam, do the same: create parent tables and insert 2–3 rows yourself.

At the end of each schema, an orange **Also asked in Sets 1–3** part covers the repeated questions that are not in the Model paper.

# 1 · Student Management

**Schema**

```
Student(StudentID PK, Name, Dept, City, CGPA)
Course(CourseID PK, CourseName, Credits)
Enrollment(EnrollID PK, StudentID FK, CourseID FK, Marks)
```

## Q1 Create the Student table with PRIMARY KEY, NOT NULL, UNIQUE, and CHECK (CGPA BETWEEN 0 AND 10) constraints. Insert the three sample student records.

**How to think**

- CREATE TABLE recipe: name, type, constraint on every column.
- Put each constraint the question names on a sensible column: PK on StudentID, NOT NULL on Name/Dept, UNIQUE on Name, CHECK on CGPA.
- Then 3 INSERTs (text in quotes, numbers without) and SELECT * to show them.

```
mysql> CREATE DATABASE college;
```

```
Query OK, 0 rows affected
```

```
mysql> USE college;
```

```
Database changed
```

```
mysql> CREATE TABLE Student (
    ->     StudentID INT PRIMARY KEY,
    ->     Name      VARCHAR(50) NOT NULL UNIQUE,
    ->     Dept      VARCHAR(20) NOT NULL,
    ->     City      VARCHAR(30),
    ->     CGPA      DECIMAL(4,2) CHECK (CGPA BETWEEN 0 AND
10)
    -> );
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Student VALUES (101, 'Arun', 'CSE',
'Chennai', 8.50);
```

```
Query OK, 1 row affected
```

```
mysql> INSERT INTO Student VALUES (102, 'Priya', 'ECE',
'Madurai', 9.10);
```

```
Query OK, 1 row affected
```

```
mysql> INSERT INTO Student VALUES (103, 'Karthik', 'CSE',
'Chennai', 7.20);
```

```
Query OK, 1 row affected
```

```
mysql> SELECT * FROM Student;
```

```
+-----------+---------+------+---------+------+
| StudentID | Name    | Dept | City    | CGPA |
+-----------+---------+------+---------+------+
|       101 | Arun    | CSE  | Chennai | 8.50 |
|       102 | Priya   | ECE  | Madurai | 9.10 |
|       103 | Karthik | CSE  | Chennai | 7.20 |
+-----------+---------+------+---------+------+
3 rows in set
```

```
mysql> DESC Student;
```

```
+-----------+--------------+------+-----+---------+-------+
| Field     | Type         | Null | Key | Default | Extra |
+-----------+--------------+------+-----+---------+-------+
| StudentID | int(11)      | NO   | PRI | NULL    |       |
| Name      | varchar(50)  | NO   | UNI | NULL    |       |
| Dept      | varchar(20)  | NO   |     | NULL    |       |
| City      | varchar(30)  | YES  |     | NULL    |       |
| CGPA      | decimal(4,2) | YES  |     | NULL    |       |
+-----------+--------------+------+-----+---------+-------+
5 rows in set
```

**Setup (not asked):** Enrollment needs a Course table to point to, so create it first (parent before child).

```
mysql> CREATE TABLE Course (
    ->     CourseID   VARCHAR(10) PRIMARY KEY,
    ->     CourseName VARCHAR(50) NOT NULL,
```

```
    ->     Credits    INT
    -> );
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Course VALUES ('C1', 'DBMS', 4), ('C2',
'Networks', 3);
```

```
Query OK, 2 rows affected
```

**Q2** **Create Enrollment with foreign keys referencing Student and Course. Insert an enrollment record for StudentID 101 and CourseID C1. Display all students from Chennai and Find the average CGPA of CSE students.**

**How to think**

- 4 sentences → 4 statements.
- Child table: FK column types must match the parents (CourseID is VARCHAR). Two FOREIGN KEY lines at the end.
- "Display ... from Chennai" → WHERE City = 'Chennai'.
- "average CGPA of CSE" → AVG(CGPA) with WHERE Dept = 'CSE'.

```
mysql> CREATE TABLE Enrollment (
    ->     EnrollID  INT PRIMARY KEY,
    ->     StudentID INT,
    ->     CourseID  VARCHAR(10),
    ->     Marks     INT,
    ->     FOREIGN KEY (StudentID) REFERENCES
Student(StudentID),
    ->     FOREIGN KEY (CourseID)  REFERENCES
Course(CourseID)
    -> );
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Enrollment VALUES (1, 101, 'C1', 85);
```

```
Query OK, 1 row affected
```

```
mysql> INSERT INTO Enrollment VALUES (2, 102, 'C2', 78);
```

```
Query OK, 1 row affected
```

```
mysql> SELECT * FROM Student WHERE City = 'Chennai';
```

```
+-----------+---------+------+---------+------+
| StudentID | Name    | Dept | City    | CGPA |
+-----------+---------+------+---------+------+
|       101 | Arun    | CSE  | Chennai | 8.50 |
|       103 | Karthik | CSE  | Chennai | 7.20 |
+-----------+---------+------+---------+------+
2 rows in set
```

```
mysql> SELECT AVG(CGPA) AS Avg_CSE_CGPA FROM Student WHERE
Dept = 'CSE';
```

```
+--------------+
| Avg_CSE_CGPA |
+--------------+
|     7.850000 |
+--------------+
1 row in set
```

**Q3** **Insert a new student and enroll the student in a course. Use COMMIT to permanently save the transaction. Display student names and course names for all enrolled students.**

**How to think**

- COMMIT recipe: SET autocommit = 0 → START TRANSACTION → the two INSERTs → COMMIT.
- Student first, then Enrollment (parent before child).
- "student names and course names" are in two tables linked through Enrollment → 3-table JOIN starting from the middle table.

```
mysql> SET autocommit = 0;
```

```
Query OK, 0 rows affected
```

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Student VALUES (104, 'Divya', 'IT',
'Coimbatore', 8.80);
```

```
Query OK, 1 row affected
```

```
mysql> INSERT INTO Enrollment VALUES (3, 104, 'C1', 90);
```

```
Query OK, 1 row affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT s.Name, c.CourseName
    -> FROM Student s
    -> JOIN Enrollment e ON s.StudentID = e.StudentID
    -> JOIN Course c     ON e.CourseID  = c.CourseID;
```

```
+-------+------------+
| Name  | CourseName |
+-------+------------+
| Arun  | DBMS       |
| Divya | DBMS       |
| Priya | Networks   |
+-------+------------+
3 rows in set
```

**Q4** Update the CGPA of a student and insert a new enrollment record. Use ROLLBACK to undo both changes. Insert a new student, create a SAVEPOINT, then insert an

**enrollment record. Roll back to the savepoint and verify which changes remain.**

### How to think

- Part 1 = ROLLBACK recipe: START TRANSACTION → UPDATE → INSERT → ROLLBACK → SELECT to prove nothing changed.
- Part 2 = SAVEPOINT recipe: A (insert student) → SAVEPOINT → B (insert enrollment) → ROLLBACK TO → COMMIT.
- "verify which changes remain" → SELECT both tables: the student stays, the enrollment is gone.

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> UPDATE Student SET CGPA = 9.50 WHERE StudentID = 103;
```

```
Query OK, 1 row affected
```

```
mysql> INSERT INTO Enrollment VALUES (4, 103, 'C2', 70);
```

```
Query OK, 1 row affected
```

```
mysql> ROLLBACK;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Student WHERE StudentID = 103;
```

```
+-----------+---------+------+---------+------+
| StudentID | Name    | Dept | City    | CGPA |
+-----------+---------+------+---------+------+
|       103 | Karthik | CSE  | Chennai | 7.20 |
+-----------+---------+------+---------+------+
1 row in set
```

```
mysql> SELECT * FROM Enrollment WHERE EnrollID = 4;
```

```
Empty set
```

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Student VALUES (105, 'Rahul', 'MECH',
'Salem', 7.90);
```

```
Query OK, 1 row affected
```

```
mysql> SAVEPOINT sp1;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Enrollment VALUES (5, 105, 'C1', 65);
```

```
Query OK, 1 row affected
```

```
mysql> ROLLBACK TO sp1;
```

```
Query OK, 0 rows affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Student WHERE StudentID = 105;
```

```
+-----------+-------+------+-------+------+
| StudentID | Name  | Dept | City  | CGPA |
+-----------+-------+------+-------+------+
|       105 | Rahul | MECH | Salem | 7.90 |
+-----------+-------+------+-------+------+
1 row in set
```

```
mysql> SELECT * FROM Enrollment WHERE StudentID = 105;
```

```
Empty set
```

**Q5** Update the marks of a student in Enrollment, create a savepoint, update the student's CGPA, and roll back to

**the savepoint. Delete an enrollment record and use ROLLBACK to restore it.**

### How to think

- SAVEPOINT recipe again: A = update marks, B = update CGPA. After ROLLBACK TO, the marks change stays and the CGPA change is undone.
- Then the restore recipe: DELETE → SELECT (gone) → ROLLBACK → SELECT (back).

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> UPDATE Enrollment SET Marks = 95 WHERE EnrollID = 1;
```

```
Query OK, 1 row affected
```

```
mysql> SAVEPOINT before_cgpa;
```

```
Query OK, 0 rows affected
```

```
mysql> UPDATE Student SET CGPA = 9.90 WHERE StudentID = 101;
```

```
Query OK, 1 row affected
```

```
mysql> ROLLBACK TO before_cgpa;
```

```
Query OK, 0 rows affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT EnrollID, Marks FROM Enrollment WHERE EnrollID
= 1;
```

```
+----------+-------+
| EnrollID | Marks |
+----------+-------+
|        1 |    95 |
+----------+-------+
1 row in set
```

```
mysql> SELECT StudentID, CGPA FROM Student WHERE StudentID = 101;
```

```
+-----------+------+
| StudentID | CGPA |
+-----------+------+
|       101 | 8.50 |
+-----------+------+
1 row in set
```

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> DELETE FROM Enrollment WHERE EnrollID = 2;
```

```
Query OK, 1 row affected
```

```
mysql> SELECT * FROM Enrollment;
```

```
+----------+-----------+----------+-------+
| EnrollID | StudentID | CourseID | Marks |
+----------+-----------+----------+-------+
|        1 |       101 | C1       |    95 |
|        3 |       104 | C1       |    90 |
+----------+-----------+----------+-------+
2 rows in set
```

```
mysql> ROLLBACK;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Enrollment;
```

```
+----------+-----------+----------+-------+
| EnrollID | StudentID | CourseID | Marks |
+----------+-----------+----------+-------+
|        1 |       101 | C1       |    95 |
|        2 |       102 | C2       |    78 |
|        3 |       104 | C1       |    90 |
+----------+-----------+----------+-------+
3 rows in set
```

**Q6** Grant SELECT permission on Student to user student_user. Grant SELECT and INSERT permissions on

## Enrollment to faculty_user. Grant SELECT, INSERT, and UPDATE permissions on Student to faculty_user.

### How to think

- Create each user first (MySQL 8 needs it).
- GRANT perms ON database.table TO 'user'@'localhost'; one line per sentence.
- SHOW GRANTS proves it.

```
mysql> SET autocommit = 1;
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE USER IF NOT EXISTS 'student_user'@'localhost'
IDENTIFIED BY 'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE USER IF NOT EXISTS 'faculty_user'@'localhost'
IDENTIFIED BY 'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT SELECT ON college.Student TO
'student_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT SELECT, INSERT ON college.Enrollment TO
'faculty_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT SELECT, INSERT, UPDATE ON college.Student TO
'faculty_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> SHOW GRANTS FOR 'faculty_user'@'localhost';
```

```
+-----------------------------------------------------------------------------------+
| Grants for faculty_user@localhost                                                 |
+-----------------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `faculty_user`@`localhost`                                  |
| GRANT SELECT, INSERT ON `college`.`Enrollment` TO `faculty_user`@`localhost`      |
| GRANT SELECT, INSERT, UPDATE ON `college`.`Student` TO `faculty_user`@`localhost` |
+-----------------------------------------------------------------------------------+
3 rows in set
```

## Q7 Revoke INSERT permission on Enrollment from faculty_user. Grant all privileges on Course to course_admin.

### How to think

- REVOKE uses FROM (GRANT uses TO).
- "all privileges" → GRANT ALL PRIVILEGES.

```
mysql> REVOKE INSERT ON college.Enrollment FROM
'faculty_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE USER IF NOT EXISTS 'course_admin'@'localhost'
IDENTIFIED BY 'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT ALL PRIVILEGES ON college.Course TO
'course_admin'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> SHOW GRANTS FOR 'faculty_user'@'localhost';
```

```
+-----------------------------------------------------------------------------------+
| Grants for faculty_user@localhost                                                 |
+-----------------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `faculty_user`@`localhost`                                  |
| GRANT SELECT, INSERT, UPDATE ON `college`.`Student` TO `faculty_user`@`localhost` |
| GRANT SELECT ON `college`.`Enrollment` TO `faculty_user`@`localhost`              |
+-----------------------------------------------------------------------------------+
3 rows in set
```

**Q8** Create a view named Student_Course_View to display StudentID, Name, CourseName, and Marks by joining Student, Course, and Enrollment and Create an index named idx_student_dept on the Dept column of the Student table.

### How to think

- Write the 3-table JOIN with exactly the columns asked, then put CREATE VIEW ... AS on top.
- Index: CREATE INDEX name ON Table(Column). The name tells you: Student, Dept.

```
mysql> CREATE VIEW Student_Course_View AS
    -> SELECT s.StudentID, s.Name, c.CourseName, e.Marks
    -> FROM Student s
    -> JOIN Enrollment e ON s.StudentID = e.StudentID
    -> JOIN Course c     ON e.CourseID  = c.CourseID;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Student_Course_View;
```

```
+-----------+-------+------------+-------+
| StudentID | Name  | CourseName | Marks |
+-----------+-------+------------+-------+
|       101 | Arun  | DBMS       |    95 |
|       104 | Divya | DBMS       |    90 |
|       102 | Priya | Networks   |    78 |
+-----------+-------+------------+-------+
3 rows in set
```

```
mysql> CREATE INDEX idx_student_dept ON Student(Dept);
```

```
Query OK, 0 rows affected
```

```
mysql> SHOW INDEX FROM Student;
```

```
+---------+------------+-----------------+--------------+-------------+
| Table   | Non_unique | Key_name        | Seq_in_index | Column_name |
+---------+------------+-----------------+--------------+-------------+
| Student |          0 | PRIMARY         |            1 | StudentID   |
| Student |          0 | Name            |            1 | Name        |
| Student |          1 | idx_student_dept |            1 | Dept        |
+---------+------------+-----------------+--------------+-------------+
3 rows in set
```

**ALSO** **Find the names of students whose CGPA is greater than the average CGPA of all students. (Sets 1, 2, 3)**

**How to think**

- "greater than average" → compare with (SELECT AVG(...)) in brackets.

```
mysql> SELECT Name FROM Student
    -> WHERE CGPA > (SELECT AVG(CGPA) FROM Student);
```

```
+-------+
| Name  |
+-------+
| Arun  |
| Priya |
| Divya |
+-------+
3 rows in set
```

# 2 · Employee Management

**Schema**

```
Employee(EmpID, EmpName, DeptID, Salary, HireDate)
Department(DeptID, DeptName, ManagerID)
```

**Q1** **Create Department and Employee tables with salary CHECK (Salary > 20000) and Insert three employees and update Meena's salary to 75000. Add a foreign key from Employee.DeptID to Department.DeptID. Try inserting an employee with DeptID 99 and explain why it fails.**

**How to think**

- Department first (parent). Employee gets CHECK (Salary > 20000). Leave the FK out of CREATE because the question says ADD it later → ALTER TABLE.
- "update Meena's salary" → UPDATE ... SET ... WHERE EmpName = 'Meena'.
- DeptID 99 is not in Department → ERROR 1452. Explain: referential integrity, the parent row must exist.

```
mysql> CREATE DATABASE company;
```

```
Query OK, 0 rows affected
```

```
mysql> USE company;
```

```
Database changed
```

```
mysql> CREATE TABLE Department (
    ->     DeptID    INT PRIMARY KEY,
    ->     DeptName  VARCHAR(30) NOT NULL,
    ->     ManagerID INT
    -> );
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE TABLE Employee (
    ->     EmpID    INT PRIMARY KEY,
    ->     EmpName  VARCHAR(50) NOT NULL,
    ->     DeptID   INT,
    ->     Salary   DECIMAL(10,2) CHECK (Salary > 20000),
    ->     HireDate DATE
    -> );
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Department VALUES (1, 'HR', 201), (2,
'IT', 202), (3, 'Sales', 204);
```

```
Query OK, 3 rows affected
```

```
mysql> INSERT INTO Employee VALUES (201, 'Ravi',   1, 45000,
'2020-06-15');
```

```
Query OK, 1 row affected
```

```
mysql> INSERT INTO Employee VALUES (202, 'Meena',  2, 60000,
'2019-03-10');
```

```
Query OK, 1 row affected
```

```
mysql> INSERT INTO Employee VALUES (203, 'Suresh', 2, 52000,
'2021-01-05');
```

```
Query OK, 1 row affected
```

```
mysql> UPDATE Employee SET Salary = 75000 WHERE EmpName =
'Meena';
```

```
Query OK, 1 row affected
```

```
mysql> SELECT * FROM Employee;
```

```
+-------+---------+--------+----------+------------+
| EmpID | EmpName | DeptID | Salary   | HireDate   |
+-------+---------+--------+----------+------------+
|   201 | Ravi    |      1 | 45000.00 | 2020-06-15 |
|   202 | Meena   |      2 | 75000.00 | 2019-03-10 |
|   203 | Suresh  |      2 | 52000.00 | 2021-01-05 |
+-------+---------+--------+----------+------------+
3 rows in set
```

```
mysql> ALTER TABLE Employee
    -> ADD CONSTRAINT fk_dept FOREIGN KEY (DeptID) REFERENCES
Department(DeptID);
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Employee VALUES (205, 'Vijay', 99, 30000,
'2022-02-02');
```

```
ERROR 1452: Cannot add or update a child row: a foreign key constraint fails (`company`.`Employee`, CONSTRAINT `fk_dept` FOREIGN KEY (`DeptID`) REFERENCES `Department` (`DeptID`))
```

**Setup (not asked):** Add two more employees so the GROUP BY questions have interesting groups.

```
mysql> INSERT INTO Employee VALUES (204, 'Anil', 3, 35000,
'2023-07-01'), (206, 'Kavya', 2, 61000, '2022-04-12');
```

```
Query OK, 2 rows affected
```

## Q2 Show departments having more than one employee using GROUP BY and HAVING. Find the highest salary in each department. Find pairs of employees who work in the same department.

### How to think

- "having more than one" → GROUP BY DeptID + HAVING COUNT(*) > 1.
- "highest ... in each" → MAX(Salary) with GROUP BY DeptID.
- "pairs ... same department" → SELF JOIN: Employee e1 JOIN Employee e2 ON same DeptID AND e1.EmpID < e2.EmpID.

```
mysql> SELECT DeptID, COUNT(*) AS EmpCount
    -> FROM Employee
```

```
    -> GROUP BY DeptID
    -> HAVING COUNT(*) > 1;
```

```
+--------+----------+
| DeptID | EmpCount |
+--------+----------+
|      2 |        3 |
+--------+----------+
1 row in set
```

```
mysql> SELECT DeptID, MAX(Salary) AS HighestSalary
    -> FROM Employee
    -> GROUP BY DeptID;
```

```
+--------+---------------+
| DeptID | HighestSalary |
+--------+---------------+
|      1 |      45000.00 |
|      2 |      75000.00 |
|      3 |      35000.00 |
+--------+---------------+
3 rows in set
```

```
mysql> SELECT e1.EmpName AS Employee1, e2.EmpName AS
Employee2, e1.DeptID
    -> FROM Employee e1
    -> JOIN Employee e2 ON e1.DeptID = e2.DeptID AND e1.EmpID
< e2.EmpID;
```

```
+-----------+-----------+--------+
| Employee1 | Employee2 | DeptID |
+-----------+-----------+--------+
| Meena     | Suresh    |      2 |
| Meena     | Kavya     |      2 |
| Suresh    | Kavya     |      2 |
+-----------+-----------+--------+
3 rows in set
```

**Q3** Increase the salary of all employees in a particular department and permanently save the changes using COMMIT. Update the salary of an employee, create a

**savepoint, update the employee's department, and roll back to the savepoint.**

### How to think

- "permanently save" = COMMIT recipe with one UPDATE ... WHERE DeptID = 2.
- Then SAVEPOINT recipe: A = salary update (kept), B = department update (undone).

```
mysql> SET autocommit = 0;
```

```
Query OK, 0 rows affected
```

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> UPDATE Employee SET Salary = Salary + 5000 WHERE
DeptID = 2;
```

```
Query OK, 3 rows affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> UPDATE Employee SET Salary = 50000 WHERE EmpID = 201;
```

```
Query OK, 1 row affected
```

```
mysql> SAVEPOINT sp1;
```

```
Query OK, 0 rows affected
```

```
mysql> UPDATE Employee SET DeptID = 2 WHERE EmpID = 201;
```

```
Query OK, 1 row affected
```

```
mysql> ROLLBACK TO sp1;
```

```
Query OK, 0 rows affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Employee;
```

```
+-------+---------+--------+----------+------------+
| EmpID | EmpName | DeptID | Salary   | HireDate   |
+-------+---------+--------+----------+------------+
|   201 | Ravi    |      1 | 50000.00 | 2020-06-15 |
|   202 | Meena   |      2 | 80000.00 | 2019-03-10 |
|   203 | Suresh  |      2 | 57000.00 | 2021-01-05 |
|   204 | Anil    |      3 | 35000.00 | 2023-07-01 |
|   206 | Kavya   |      2 | 66000.00 | 2022-04-12 |
+-------+---------+--------+----------+------------+
5 rows in set
```

**Q4** **Delete an employee record and use ROLLBACK to undo the deletion. Insert a new department and employee. Use COMMIT to save both operations.**

### How to think

- Restore recipe: DELETE → SELECT → ROLLBACK → SELECT.
- COMMIT recipe: department first (parent), then the employee in it.

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> DELETE FROM Employee WHERE EmpID = 203;
```

```
Query OK, 1 row affected
```

```
mysql> SELECT COUNT(*) FROM Employee;
```

```
+----------+
| COUNT(*) |
+----------+
|        4 |
+----------+
1 row in set
```

```
mysql> ROLLBACK;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT COUNT(*) FROM Employee;
```

```
+----------+
| COUNT(*) |
+----------+
|        5 |
+----------+
1 row in set
```

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Department VALUES (4, 'Finance', 207);
```

```
Query OK, 1 row affected
```

```
mysql> INSERT INTO Employee VALUES (207, 'Nisha', 4, 40000,
'2024-01-10');
```

```
Query OK, 1 row affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Employee WHERE DeptID = 4;
```

```
+-------+---------+--------+----------+------------+
| EmpID | EmpName | DeptID | Salary   | HireDate   |
+-------+---------+--------+----------+------------+
|   207 | Nisha   |      4 | 40000.00 | 2024-01-10 |
+-------+---------+--------+----------+------------+
1 row in set
```

**Q5** Increase the salary of employees by 10%, create a savepoint, increase it by another 5%, and roll back to the first savepoint.

### How to think

- +10% → Salary * 1.10, +5% → Salary * 1.05.
- After ROLLBACK TO, only the 10% stays.

```
mysql> SELECT EmpID, Salary FROM Employee;
```

```
+-------+----------+
| EmpID | Salary   |
+-------+----------+
|   201 | 50000.00 |
|   202 | 80000.00 |
|   203 | 57000.00 |
|   204 | 35000.00 |
|   206 | 66000.00 |
|   207 | 40000.00 |
+-------+----------+
6 rows in set
```

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> UPDATE Employee SET Salary = Salary * 1.10;
```

```
Query OK, 6 rows affected
```

```
mysql> SAVEPOINT after_10;
```

```
Query OK, 0 rows affected
```

```
mysql> UPDATE Employee SET Salary = Salary * 1.05;
```

```
Query OK, 6 rows affected
```

```
mysql> ROLLBACK TO after_10;
```

```
Query OK, 0 rows affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT EmpID, Salary FROM Employee;
```

```
+-------+----------+
| EmpID | Salary   |
+-------+----------+
|   201 | 55000.00 |
|   202 | 88000.00 |
|   203 | 62700.00 |
|   204 | 38500.00 |
|   206 | 72600.00 |
|   207 | 44000.00 |
+-------+----------+
6 rows in set
```

**Q6** Grant SELECT permission on Employee to hr_user. Grant SELECT and UPDATE permissions on Employee to manager_user. Grant INSERT permission on Employee to hr_user.

**How to think**

- One GRANT per sentence; create the users first.

```
mysql> SET autocommit = 1;
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE USER IF NOT EXISTS 'hr_user'@'localhost'
IDENTIFIED BY 'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE USER IF NOT EXISTS 'manager_user'@'localhost'
IDENTIFIED BY 'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT SELECT ON company.Employee TO
'hr_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT SELECT, UPDATE ON company.Employee TO
'manager_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT INSERT ON company.Employee TO
'hr_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> SHOW GRANTS FOR 'hr_user'@'localhost';
```

```
+-----------------------------------------------------------------------+
| Grants for hr_user@localhost                                          |
+-----------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `hr_user`@`localhost`                           |
| GRANT SELECT, INSERT ON `company`.`Employee` TO `hr_user`@`localhost` |
+-----------------------------------------------------------------------+
2 rows in set
```

## Q7 Revoke UPDATE permission on Employee from manager_user. Grant SELECT permission on Department to employee_user.

### How to think

- REVOKE ... FROM; new user employee_user needs CREATE USER first.

```
mysql> REVOKE UPDATE ON company.Employee FROM
'manager_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE USER IF NOT EXISTS 'employee_user'@'localhost'
IDENTIFIED BY 'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT SELECT ON company.Department TO
'employee_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> SHOW GRANTS FOR 'manager_user'@'localhost';
```

```
+-------------------------------------------------------------------+
| Grants for manager_user@localhost                                 |
+-------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `manager_user`@`localhost`                  |
| GRANT SELECT ON `company`.`Employee` TO `manager_user`@`localhost` |
+-------------------------------------------------------------------+
2 rows in set
```

**Q8** **Create a view named Employee_Department_View to display EmpID, EmpName, Salary, and DeptName by joining Employee and Department and Create an index named idx_employee_salary on the Salary column of the Employee table.**

**How to think**

- 2-table JOIN on DeptID with the four columns asked, CREATE VIEW on top.
- Index on Employee(Salary).

```
mysql> CREATE VIEW Employee_Department_View AS
    -> SELECT e.EmpID, e.EmpName, e.Salary, d.DeptName
    -> FROM Employee e
    -> JOIN Department d ON e.DeptID = d.DeptID;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Employee_Department_View;
```

```
+-------+---------+----------+----------+
| EmpID | EmpName | Salary   | DeptName |
+-------+---------+----------+----------+
|   201 | Ravi    | 55000.00 | HR       |
|   202 | Meena   | 88000.00 | IT       |
|   203 | Suresh  | 62700.00 | IT       |
|   204 | Anil    | 38500.00 | Sales    |
|   206 | Kavya   | 72600.00 | IT       |
|   207 | Nisha   | 44000.00 | Finance  |
+-------+---------+----------+----------+
6 rows in set
```

```
mysql> CREATE INDEX idx_employee_salary ON Employee(Salary);
```

```
Query OK, 0 rows affected
```

---

**ALSO** Display the EmpID, EmpName, and Salary of employees whose salary is greater than the average salary of all employees. (ALL 4 papers)

**How to think**

- Average subquery.

```
mysql> SELECT EmpID, EmpName, Salary FROM Employee
    -> WHERE Salary > (SELECT AVG(Salary) FROM Employee);
```

```
+-------+---------+----------+
| EmpID | EmpName | Salary   |
+-------+---------+----------+
|   203 | Suresh  | 62700.00 |
|   206 | Kavya   | 72600.00 |
|   202 | Meena   | 88000.00 |
+-------+---------+----------+
3 rows in set
```

# 3 · Railway Reservation

**Schema**

```
Passenger(PassengerID, Name, Age)
Train(TrainID, TrainName, Source, Destination)
Booking(BookingID, PassengerID, TrainID, SeatsBooked, Fare)
```

**Q1** Create the Train table with TrainName UNIQUE. Insert two trains and delete the train whose destination is Coimbatore.

**How to think**

- UNIQUE on TrainName. Two INSERTs. DELETE ... WHERE Destination = 'Coimbatore'.
- SELECT before and after to show it.

```
mysql> CREATE DATABASE railway;
```

```
Query OK, 0 rows affected
```

```
mysql> USE railway;
```

```
Database changed
```

```
mysql> CREATE TABLE Train (
    ->     TrainID     INT PRIMARY KEY,
    ->     TrainName   VARCHAR(50) UNIQUE,
    ->     Source      VARCHAR(30),
    ->     Destination VARCHAR(30)
    -> );
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Train VALUES (1, 'Vaigai Express',
'Chennai', 'Madurai');
```

```
Query OK, 1 row affected
```

```
mysql> INSERT INTO Train VALUES (2, 'Kovai Express',
'Chennai', 'Coimbatore');
```

```
Query OK, 1 row affected
```

```
mysql> SELECT * FROM Train;
```

```
+---------+----------------+---------+-------------+
| TrainID | TrainName      | Source  | Destination |
+---------+----------------+---------+-------------+
|       1 | Vaigai Express | Chennai | Madurai     |
|       2 | Kovai Express  | Chennai | Coimbatore  |
+---------+----------------+---------+-------------+
2 rows in set
```

```
mysql> DELETE FROM Train WHERE Destination = 'Coimbatore';
```

```
Query OK, 1 row affected
```

```
mysql> SELECT * FROM Train;
```

```
+---------+----------------+---------+-------------+
| TrainID | TrainName      | Source  | Destination |
+---------+----------------+---------+-------------+
|       1 | Vaigai Express | Chennai | Madurai     |
+---------+----------------+---------+-------------+
1 row in set
```

**Setup (not asked):** Booking needs Passenger rows and two trains, so add them back.

```
mysql> CREATE TABLE Passenger (
    ->     PassengerID INT PRIMARY KEY,
    ->     Name        VARCHAR(50) NOT NULL,
    ->     Age         INT
    -> );
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Passenger VALUES (1, 'Arun', 25), (2,
'Priya', 30), (3, 'Kumar', 40);
```

```
Query OK, 3 rows affected
```

```
mysql> INSERT INTO Train VALUES (2, 'Kovai Express',
'Chennai', 'Coimbatore');
```

```
Query OK, 1 row affected
```

## Q2 Create Booking with foreign keys to Passenger and Train. Add ON DELETE CASCADE on Booking.TrainID and explain what happens when a train is deleted.

### How to think

- Two FOREIGN KEY lines; put ON DELETE CASCADE on the TrainID one.
- Explain: deleting a train automatically deletes all its bookings. Prove it: delete train 2, then SELECT * FROM Booking.

```
mysql> CREATE TABLE Booking (
    ->     BookingID   INT PRIMARY KEY,
    ->     PassengerID INT,
    ->     TrainID     INT,
    ->     SeatsBooked INT,
    ->     Fare        DECIMAL(8,2),
    ->     FOREIGN KEY (PassengerID) REFERENCES
Passenger(PassengerID),
    ->     FOREIGN KEY (TrainID) REFERENCES Train(TrainID) ON
DELETE CASCADE
    -> );
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Booking VALUES (1, 1, 1, 2, 500), (2, 2,
1, 1, 300), (3, 1, 2, 3, 900);
```

```
Query OK, 3 rows affected
```

```
mysql> DELETE FROM Train WHERE TrainID = 2;
```

```
Query OK, 1 row affected
```

```
mysql> SELECT * FROM Booking;
```

```
+-----------+-------------+---------+-------------+--------+
| BookingID | PassengerID | TrainID | SeatsBooked | Fare   |
+-----------+-------------+---------+-------------+--------+
|         1 |           1 |       1 |           2 | 500.00 |
|         2 |           2 |       1 |           1 | 300.00 |
+-----------+-------------+---------+-------------+--------+
2 rows in set
```

**Setup (not asked):** Put train 2 and its booking back, and add one more booking.

```
mysql> INSERT INTO Train VALUES (2, 'Kovai Express',
'Chennai', 'Coimbatore');
```

```
Query OK, 1 row affected
```

```
mysql> INSERT INTO Booking VALUES (3, 1, 2, 3, 900), (4, 2,
2, 1, 450);
```

```
Query OK, 2 rows affected
```

## Q3 Show all passengers and their bookings; include passengers who have not booked any train.

### How to think

- "include ... who have not" → LEFT JOIN with Passenger first. Kumar shows NULLs.

```
mysql> SELECT p.PassengerID, p.Name, b.BookingID, b.TrainID,
b.Fare
```

```
    -> FROM Passenger p
    -> LEFT JOIN Booking b ON p.PassengerID = b.PassengerID;
```

```
+-------------+-------+-----------+---------+--------+
| PassengerID | Name  | BookingID | TrainID | Fare   |
+-------------+-------+-----------+---------+--------+
|           1 | Arun  |         1 |       1 | 500.00 |
|           2 | Priya |         2 |       1 | 300.00 |
|           1 | Arun  |         3 |       2 | 900.00 |
|           2 | Priya |         4 |       2 | 450.00 |
|           3 | Kumar | NULL      | NULL    | NULL   |
+-------------+-------+-----------+---------+--------+
5 rows in set
```

**Q4** Find bookings whose fare is greater than the average fare of that same train.

**How to think**

- "of that same train" → correlated subquery: inner WHERE b2.TrainID = b1.TrainID.

```
mysql> SELECT * FROM Booking b1
    -> WHERE Fare > (SELECT AVG(Fare) FROM Booking b2 WHERE
b2.TrainID = b1.TrainID);
```

```
+-----------+-------------+---------+-------------+--------+
| BookingID | PassengerID | TrainID | SeatsBooked | Fare   |
+-----------+-------------+---------+-------------+--------+
|         1 |           1 |       1 |           2 | 500.00 |
|         3 |           1 |       2 |           3 | 900.00 |
+-----------+-------------+---------+-------------+--------+
2 rows in set
```

**Q5** Insert a new passenger and create a booking for the passenger. Use COMMIT. Calculate the total fare collected for each train. Update the number of seats

**booked and fare for a booking. Use ROLLBACK to cancel the changes.**

### How to think

- COMMIT recipe (passenger then booking).
- "total ... for each train" → SUM(Fare) GROUP BY TrainID.
- ROLLBACK recipe on one UPDATE that changes 2 columns (SET a = ..., b = ...).

```
mysql> SET autocommit = 0;
```

```
Query OK, 0 rows affected
```

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Passenger VALUES (4, 'Divya', 22);
```

```
Query OK, 1 row affected
```

```
mysql> INSERT INTO Booking VALUES (5, 4, 1, 2, 600);
```

```
Query OK, 1 row affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT TrainID, SUM(Fare) AS TotalFare FROM Booking
GROUP BY TrainID;
```

```
+---------+-----------+
| TrainID | TotalFare |
+---------+-----------+
|       1 |   1400.00 |
|       2 |   1350.00 |
+---------+-----------+
2 rows in set
```

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> UPDATE Booking SET SeatsBooked = 5, Fare = 1500 WHERE
BookingID = 1;
```

```
Query OK, 1 row affected
```

```
mysql> ROLLBACK;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Booking WHERE BookingID = 1;
```

```
+-----------+-------------+---------+-------------+--------+
| BookingID | PassengerID | TrainID | SeatsBooked | Fare   |
+-----------+-------------+---------+-------------+--------+
|         1 |           1 |       1 |           2 | 500.00 |
+-----------+-------------+---------+-------------+--------+
1 row in set
```

**Q6** **Insert a passenger, create a SAVEPOINT, make a booking, and roll back to the savepoint. Delete a booking and use ROLLBACK to restore it. Modify the fare of a booking, create a savepoint, modify the seats booked, and roll back to the savepoint.**

**How to think**

- Three small recipes in a row: SAVEPOINT, restore, SAVEPOINT.

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Passenger VALUES (5, 'Rahul', 28);
```

```
Query OK, 1 row affected
```

```
mysql> SAVEPOINT sp1;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Booking VALUES (6, 5, 2, 1, 450);
```

```
Query OK, 1 row affected
```

```
mysql> ROLLBACK TO sp1;
```

```
Query OK, 0 rows affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> DELETE FROM Booking WHERE BookingID = 2;
```

```
Query OK, 1 row affected
```

```
mysql> ROLLBACK;
```

```
Query OK, 0 rows affected
```

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> UPDATE Booking SET Fare = 550 WHERE BookingID = 1;
```

```
Query OK, 1 row affected
```

```
mysql> SAVEPOINT sp2;
```

```
Query OK, 0 rows affected
```

```
mysql> UPDATE Booking SET SeatsBooked = 4 WHERE BookingID =
1;
```

```
Query OK, 1 row affected
```

```
mysql> ROLLBACK TO sp2;
```

```
Query OK, 0 rows affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Booking;
```

```
+-----------+-------------+---------+-------------+--------+
| BookingID | PassengerID | TrainID | SeatsBooked | Fare   |
+-----------+-------------+---------+-------------+--------+
|         1 |           1 |       1 |           2 | 550.00 |
|         2 |           2 |       1 |           1 | 300.00 |
|         3 |           1 |       2 |           3 | 900.00 |
|         4 |           2 |       2 |           1 | 450.00 |
|         5 |           4 |       1 |           2 | 600.00 |
+-----------+-------------+---------+-------------+--------+
5 rows in set
```

**Q7** Grant SELECT permission on Train to reservation_user. Grant SELECT and INSERT permissions on Booking to booking_user. Grant SELECT, INSERT, and UPDATE permissions on Passenger to reservation_user.

**How to think**

- Create the two users, then one GRANT per sentence.

```
mysql> SET autocommit = 1;
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE USER IF NOT EXISTS
'reservation_user'@'localhost' IDENTIFIED BY 'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE USER IF NOT EXISTS 'booking_user'@'localhost'
IDENTIFIED BY 'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT SELECT ON railway.Train TO
'reservation_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT SELECT, INSERT ON railway.Booking TO
'booking_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT SELECT, INSERT, UPDATE ON railway.Passenger TO
'reservation_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> SHOW GRANTS FOR 'reservation_user'@'localhost';
```

```
+-----------------------------------------------------------------------------------------+
| Grants for reservation_user@localhost                                                   |
+-----------------------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `reservation_user`@`localhost`                                    |
| GRANT SELECT, INSERT, UPDATE ON `railway`.`Passenger` TO `reservation_user`@`localhost` |
| GRANT SELECT ON `railway`.`Train` TO `reservation_user`@`localhost`                     |
+-----------------------------------------------------------------------------------------+
3 rows in set
```

**Q8** Revoke INSERT permission on Booking from booking_user. Grant all privileges on Booking to railway_admin.

**How to think**

- REVOKE ... FROM, GRANT ALL PRIVILEGES ... TO.

```
mysql> REVOKE INSERT ON railway.Booking FROM
'booking_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE USER IF NOT EXISTS 'railway_admin'@'localhost'
IDENTIFIED BY 'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT ALL PRIVILEGES ON railway.Booking TO
'railway_admin'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> SHOW GRANTS FOR 'booking_user'@'localhost';
```

```
+---------------------------------------------------------------------+
| Grants for booking_user@localhost                                   |
+---------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `booking_user`@`localhost`                    |
| GRANT SELECT ON `railway`.`Booking` TO `booking_user`@`localhost`   |
+---------------------------------------------------------------------+
2 rows in set
```

**Q9** **Create a view named Booking_Details_View to display BookingID, passenger Name, TrainName, SeatsBooked, and Fare by joining Passenger, Train, and Booking. Create an index named idx_train_destination on the Destination column of the Train table.**

**How to think**

- Middle table Booking, JOIN Passenger and Train.
- Index on Train(Destination).

```
mysql> CREATE VIEW Booking_Details_View AS
    -> SELECT b.BookingID, p.Name, t.TrainName,
b.SeatsBooked, b.Fare
    -> FROM Booking b
    -> JOIN Passenger p ON b.PassengerID = p.PassengerID
    -> JOIN Train t     ON b.TrainID     = t.TrainID;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Booking_Details_View;
```

```
+-----------+-------+---------------+-------------+--------+
| BookingID | Name  | TrainName     | SeatsBooked | Fare   |
+-----------+-------+---------------+-------------+--------+
|         1 | Arun  | Vaigai Express |           2 | 550.00 |
|         2 | Priya | Vaigai Express |           1 | 300.00 |
|         3 | Arun  | Kovai Express  |           3 | 900.00 |
|         4 | Priya | Kovai Express  |           1 | 450.00 |
|         5 | Divya | Vaigai Express |           2 | 600.00 |
+-----------+-------+---------------+-------------+--------+
5 rows in set
```

```
mysql> CREATE INDEX idx_train_destination ON
Train(Destination);
```

```
Query OK, 0 rows affected
```

---

**ALSO** **Display the names of passengers who have made at least one booking. (ALL 4 papers)**

### How to think

- "at least one" → IN (SELECT PassengerID FROM Booking).

```
mysql> SELECT Name FROM Passenger
    -> WHERE PassengerID IN (SELECT PassengerID FROM
Booking);
```

```
+-------+
| Name  |
+-------+
| Arun  |
| Priya |
| Divya |
+-------+
3 rows in set
```

# 4 · Hospital Management

**Schema**

```
Patient(PatientID, PatientName, City)
Doctor(DoctorID, DoctorName, Specialization)
Appointment(AppID, PatientID, DoctorID, VisitDate, Fee)
```

**Setup (not asked):** Appointment needs a Patient table too.

```
mysql> CREATE DATABASE hospital;
```

```
Query OK, 0 rows affected
```

```
mysql> USE hospital;
```

```
Database changed
```

```
mysql> CREATE TABLE Patient (
    ->     PatientID   INT PRIMARY KEY,
    ->     PatientName VARCHAR(50) NOT NULL,
    ->     City        VARCHAR(30)
    -> );
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Patient VALUES (1, 'Arun', 'Chennai'), (2,
'Priya', 'Madurai');
```

```
Query OK, 2 rows affected
```

## Q1 Create the Doctor table with Specialization NOT NULL. Create Appointment with foreign keys to Patient and Doctor.

**How to think**

- NOT NULL on Specialization. Appointment = middle table with two FOREIGN KEY lines.
- Bonus proof: inserting a doctor without specialization fails.

```
mysql> CREATE TABLE Doctor (
    ->     DoctorID       INT PRIMARY KEY,
    ->     DoctorName     VARCHAR(50) NOT NULL,
    ->     Specialization VARCHAR(40) NOT NULL
    -> );
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE TABLE Appointment (
    ->     AppID     INT PRIMARY KEY,
    ->     PatientID INT,
    ->     DoctorID  INT,
    ->     VisitDate DATE,
    ->     Fee       DECIMAL(8,2),
    ->     FOREIGN KEY (PatientID) REFERENCES
Patient(PatientID),
    ->     FOREIGN KEY (DoctorID)  REFERENCES
Doctor(DoctorID)
    -> );
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Doctor VALUES (1, 'Dr. Ravi',
'Cardiology'), (2, 'Dr. Meena', 'Neurology'), (3, 'Dr.
Kumar', 'Ortho');
```

```
Query OK, 3 rows affected
```

```
mysql> INSERT INTO Doctor (DoctorID, DoctorName) VALUES (4,
'Dr. X');
```

```
ERROR 1364: Field 'Specialization' doesn't have a default value
```

```
mysql> INSERT INTO Appointment VALUES (1, 1, 1, '2025-01-10',
500), (2, 2, 1, '2025-01-11', 500), (3, 1, 2, '2025-01-12',
800);
```

```
Query OK, 3 rows affected
```

## Q2 Count the number of appointments handled by each doctor.

### How to think

- "number of ... each doctor" → COUNT with GROUP BY. JOIN Doctor to show names (LEFT JOIN also shows doctors with 0).

```
mysql> SELECT d.DoctorName, COUNT(a.AppID) AS
TotalAppointments
    -> FROM Doctor d
    -> LEFT JOIN Appointment a ON d.DoctorID = a.DoctorID
    -> GROUP BY d.DoctorID, d.DoctorName;
```

```
+------------+------------------+
| DoctorName | TotalAppointments |
+------------+------------------+
| Dr. Ravi   |                2 |
| Dr. Meena  |                1 |
| Dr. Kumar  |                0 |
+------------+------------------+
3 rows in set
```

## Q3 Display patient names and doctor names for all appointments.

### How to think

- 3-table JOIN from the middle table Appointment.

```
mysql> SELECT p.PatientName, d.DoctorName
    -> FROM Appointment a
    -> JOIN Patient p ON a.PatientID = p.PatientID
    -> JOIN Doctor d  ON a.DoctorID  = d.DoctorID;
```

```
+-------------+------------+
| PatientName | DoctorName |
+-------------+------------+
| Arun        | Dr. Ravi   |
| Priya       | Dr. Ravi   |
| Arun        | Dr. Meena  |
+-------------+------------+
3 rows in set
```

**Q4** Insert a new patient and create an appointment for the patient. Use COMMIT. Update the appointment fee and use ROLLBACK to undo the change.

**How to think**

- COMMIT recipe, then ROLLBACK recipe with a SELECT to prove the fee is unchanged.

```
mysql> SET autocommit = 0;
```

```
Query OK, 0 rows affected
```

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Patient VALUES (3, 'Kavya', 'Salem');
```

```
Query OK, 1 row affected
```

```
mysql> INSERT INTO Appointment VALUES (4, 3, 3, '2025-02-01',
600);
```

```
Query OK, 1 row affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> UPDATE Appointment SET Fee = 1000 WHERE AppID = 4;
```

```
Query OK, 1 row affected
```

```
mysql> ROLLBACK;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Appointment WHERE AppID = 4;
+-------+-----------+----------+------------+--------+
| AppID | PatientID | DoctorID | VisitDate  | Fee    |
+-------+-----------+----------+------------+--------+
|     4 |         3 |        3 | 2025-02-01 | 600.00 |
+-------+-----------+----------+------------+--------+
1 row in set
```

**Q5** **Insert a new patient, create a savepoint, insert an appointment, and roll back to the savepoint. Delete an appointment and use ROLLBACK to restore it.**

**How to think**

- SAVEPOINT recipe (patient kept, appointment undone), then restore recipe.

```
mysql> START TRANSACTION;
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Patient VALUES (4, 'Rahul', 'Trichy');
Query OK, 1 row affected
```

```
mysql> SAVEPOINT sp1;
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Appointment VALUES (5, 4, 2, '2025-02-05',
800);
Query OK, 1 row affected
```

```
mysql> ROLLBACK TO sp1;
Query OK, 0 rows affected
```

```
mysql> COMMIT;
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Appointment WHERE PatientID = 4;
```

```
Empty set
```

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> DELETE FROM Appointment WHERE AppID = 3;
```

```
Query OK, 1 row affected
```

```
mysql> ROLLBACK;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT COUNT(*) FROM Appointment;
```

```
+----------+
| COUNT(*) |
+----------+
|        4 |
+----------+
1 row in set
```

## Q6 Update a doctor's specialization, create a savepoint, update an appointment fee, and roll back to the savepoint.

### How to think

- A = specialization (kept), B = fee (undone).

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> UPDATE Doctor SET Specialization = 'Cardiac Surgery'
WHERE DoctorID = 1;
```

```
Query OK, 1 row affected
```

```
mysql> SAVEPOINT sp2;
```

```
Query OK, 0 rows affected
```

```
mysql> UPDATE Appointment SET Fee = 900 WHERE AppID = 1;
```

```
Query OK, 1 row affected
```

```
mysql> ROLLBACK TO sp2;
```

```
Query OK, 0 rows affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Doctor WHERE DoctorID = 1;
```

```
+----------+------------+-----------------+
| DoctorID | DoctorName | Specialization  |
+----------+------------+-----------------+
|        1 | Dr. Ravi   | Cardiac Surgery |
+----------+------------+-----------------+
1 row in set
```

```
mysql> SELECT AppID, Fee FROM Appointment WHERE AppID = 1;
```

```
+-------+--------+
| AppID | Fee    |
+-------+--------+
|     1 | 500.00 |
+-------+--------+
1 row in set
```

**Q7** **Grant SELECT permission on Doctor to receptionist. Grant SELECT and INSERT permissions on Appointment to receptionist. Grant SELECT, INSERT, and UPDATE permissions on Patient to hospital_staff.**

**How to think**

- Create users, one GRANT per sentence.

```
mysql> SET autocommit = 1;
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE USER IF NOT EXISTS 'receptionist'@'localhost'
```

```
IDENTIFIED BY 'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE USER IF NOT EXISTS 'hospital_staff'@'localhost'
IDENTIFIED BY 'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT SELECT ON hospital.Doctor TO
'receptionist'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT SELECT, INSERT ON hospital.Appointment TO
'receptionist'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT SELECT, INSERT, UPDATE ON hospital.Patient TO
'hospital_staff'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> SHOW GRANTS FOR 'receptionist'@'localhost';
```

```
+-----------------------------------------------------------------------------+
| Grants for receptionist@localhost                                           |
+-----------------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `receptionist`@`localhost`                            |
| GRANT SELECT ON `hospital`.`Doctor` TO `receptionist`@`localhost`           |
| GRANT SELECT, INSERT ON `hospital`.`Appointment` TO `receptionist`@`localhost` |
+-----------------------------------------------------------------------------+
3 rows in set
```

## Q8 Revoke INSERT permission on Appointment from receptionist. Grant SELECT permission on Doctor to doctor_user.

**How to think**

- REVOKE ... FROM; create doctor_user first.

```
mysql> REVOKE INSERT ON hospital.Appointment FROM
'receptionist'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE USER IF NOT EXISTS 'doctor_user'@'localhost'
IDENTIFIED BY 'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT SELECT ON hospital.Doctor TO
'doctor_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> SHOW GRANTS FOR 'receptionist'@'localhost';
```

```
+-----------------------------------------------------------------------+
| Grants for receptionist@localhost                                     |
+-----------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `receptionist`@`localhost`                      |
| GRANT SELECT ON `hospital`.`Doctor` TO `receptionist`@`localhost`     |
| GRANT SELECT ON `hospital`.`Appointment` TO `receptionist`@`localhost` |
+-----------------------------------------------------------------------+
3 rows in set
```

**Q9** **Create a view named Appointment_Details_View to display AppID, patient PatientName, doctor DoctorName, Specialization, VisitDate, and Fee by joining Patient, Doctor, and Appointment. Create an index named idx_doctor_specialization on the Specialization column of the Doctor table.**

### How to think

- Middle table Appointment + two JOINs. Index on Doctor(Specialization).

```
mysql> CREATE VIEW Appointment_Details_View AS
    -> SELECT a.AppID, p.PatientName, d.DoctorName,
d.Specialization, a.VisitDate, a.Fee
    -> FROM Appointment a
    -> JOIN Patient p ON a.PatientID = p.PatientID
    -> JOIN Doctor d  ON a.DoctorID  = d.DoctorID;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Appointment_Details_View;
```

```
+-------+-------------+------------+----------------+------------+--------+
| AppID | PatientName | DoctorName | Specialization | VisitDate  | Fee    |
+-------+-------------+------------+----------------+------------+--------+
|     1 | Arun        | Dr. Ravi   | Cardiac Surgery | 2025-01-10 | 500.00 |
|     2 | Priya       | Dr. Ravi   | Cardiac Surgery | 2025-01-11 | 500.00 |
|     3 | Arun        | Dr. Meena  | Neurology       | 2025-01-12 | 800.00 |
|     4 | Kavya       | Dr. Kumar  | Ortho           | 2025-02-01 | 600.00 |
+-------+-------------+------------+----------------+------------+--------+
4 rows in set
```

```
mysql> CREATE INDEX idx_doctor_specialization ON
Doctor(Specialization);
```

```
Query OK, 0 rows affected
```

---

**ALSO** Display the details of doctors who have at least one appointment. (ALL 4 papers)

**How to think**

- "at least one" → IN.

```
mysql> SELECT * FROM Doctor
    -> WHERE DoctorID IN (SELECT DoctorID FROM Appointment);
```

```
+----------+------------+-----------------+
| DoctorID | DoctorName | Specialization  |
+----------+------------+-----------------+
|        1 | Dr. Ravi   | Cardiac Surgery |
|        2 | Dr. Meena  | Neurology       |
|        3 | Dr. Kumar  | Ortho           |
+----------+------------+-----------------+
3 rows in set
```

# 5 · Bank Management

**Schema**

```
Customer(CustomerID, CustomerName, City)
Account(AccountNo, CustomerID, AccountType, Balance)
Transaction(TxnID, AccountNo, Amount, TxnType)
```

**Setup (not asked):** Account's FK needs the Customer table.

```
mysql> CREATE DATABASE bank;
```

```
Query OK, 0 rows affected
```

```
mysql> USE bank;
```

```
Database changed
```

```
mysql> CREATE TABLE Customer (
    ->     CustomerID   INT PRIMARY KEY,
    ->     CustomerName VARCHAR(50) NOT NULL,
    ->     City         VARCHAR(30)
    -> );
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Customer VALUES (1, 'Arun', 'Chennai'),
(2, 'Priya', 'Madurai'), (3, 'Kumar', 'Salem');
```

```
Query OK, 3 rows affected
```

**Q1** **Create the Account table with CHECK (Balance >= 1000). Add a foreign key from Account.CustomerID to Customer.CustomerID.**

**How to think**

- CHECK on Balance in CREATE. "Add a foreign key" → ALTER TABLE ... ADD CONSTRAINT.
- Proof: a balance of 500 is rejected by the CHECK.

```
mysql> CREATE TABLE Account (
    ->     AccountNo   INT PRIMARY KEY,
    ->     CustomerID  INT,
    ->     AccountType VARCHAR(20),
    ->     Balance     DECIMAL(12,2) CHECK (Balance >= 1000)
    -> );
```

```
Query OK, 0 rows affected
```

```
mysql> ALTER TABLE Account
    -> ADD CONSTRAINT fk_cust FOREIGN KEY (CustomerID)
REFERENCES Customer(CustomerID);
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Account VALUES (1001, 1, 'Savings',
50000), (1002, 2, 'Current', 80000), (1003, 3, 'Savings',
20000);
```

```
Query OK, 3 rows affected
```

```
mysql> INSERT INTO Account VALUES (1009, 1, 'Savings', 500);
```

```
ERROR 4025: CONSTRAINT `Account.Balance` failed for `bank`.`Account`
```

**Setup (not asked):** The Transaction table (keyword → backticks).

```
mysql> CREATE TABLE `Transaction` (
    ->     TxnID     INT PRIMARY KEY,
    ->     AccountNo INT,
    ->     Amount    DECIMAL(12,2),
    ->     TxnType   VARCHAR(10),
    ->     FOREIGN KEY (AccountNo) REFERENCES
Account(AccountNo)
    -> );
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO `Transaction` VALUES (1, 1001, 5000,
'Credit');
```

```
Query OK, 1 row affected
```

## Q2 Find customers whose account balance is greater than the average balance.

### How to think

- Customer names live in Customer, balance in Account → JOIN, plus the average subquery.

```
mysql> SELECT c.CustomerName, a.Balance
    -> FROM Customer c
    -> JOIN Account a ON c.CustomerID = a.CustomerID
    -> WHERE a.Balance > (SELECT AVG(Balance) FROM Account);
```

```
+--------------+----------+
| CustomerName | Balance  |
+--------------+----------+
| Priya        | 80000.00 |
+--------------+----------+
1 row in set
```

## Q3 Find accounts whose balance is equal to the maximum balance in the bank.

### How to think

- "equal to maximum" → = (SELECT MAX(Balance) ...).

```
mysql> SELECT * FROM Account
    -> WHERE Balance = (SELECT MAX(Balance) FROM Account);
```

```
+-----------+------------+-------------+----------+
| AccountNo | CustomerID | AccountType | Balance  |
+-----------+------------+-------------+----------+
|      1002 |          2 | Current     | 80000.00 |
+-----------+------------+-------------+----------+
1 row in set
```

**Q4** Insert a new customer and create an account for the customer. Use COMMIT. Deduct ₹5,000 from an account balance and record the transaction. Use COMMIT.

**How to think**

- Two COMMIT recipes.
- Deduct → SET Balance = Balance - 5000, and record → INSERT a 'Debit' row.

```
mysql> SET autocommit = 0;
```

```
Query OK, 0 rows affected
```

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Customer VALUES (4, 'Divya', 'Trichy');
```

```
Query OK, 1 row affected
```

```
mysql> INSERT INTO Account VALUES (1004, 4, 'Savings',
15000);
```

```
Query OK, 1 row affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> UPDATE Account SET Balance = Balance - 5000 WHERE
AccountNo = 1001;
```

```
Query OK, 1 row affected
```

```
mysql> INSERT INTO `Transaction` VALUES (2, 1001, 5000,
'Debit');
```

```
Query OK, 1 row affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Account;
```

```
+-----------+------------+-------------+----------+
| AccountNo | CustomerID | AccountType | Balance  |
+-----------+------------+-------------+----------+
|      1001 |          1 | Savings     | 45000.00 |
|      1002 |          2 | Current     | 80000.00 |
|      1003 |          3 | Savings     | 20000.00 |
|      1004 |          4 | Savings     | 15000.00 |
+-----------+------------+-------------+----------+
4 rows in set
```

**Q5** **Transfer ₹10,000 from one account to another account and record both transactions. Use ROLLBACK if any operation fails. Deduct an amount from an account, create a savepoint, insert the transaction record, and roll back to the savepoint.**

### How to think

- Transfer = − on one account, + on the other, 2 transaction rows, COMMIT (say: if any step errors, type ROLLBACK instead, all or nothing).
- Then SAVEPOINT recipe: A = deduct (kept), B = transaction row (undone).

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> UPDATE Account SET Balance = Balance - 10000 WHERE
AccountNo = 1002;
```

```
Query OK, 1 row affected
```

```
mysql> UPDATE Account SET Balance = Balance + 10000 WHERE
AccountNo = 1003;
```

```
Query OK, 1 row affected
```

```
mysql> INSERT INTO `Transaction` VALUES (3, 1002, 10000,
'Debit');
```

```
Query OK, 1 row affected
```

```
mysql> INSERT INTO `Transaction` VALUES (4, 1003, 10000,
'Credit');
```

```
Query OK, 1 row affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> UPDATE Account SET Balance = Balance - 2000 WHERE
AccountNo = 1003;
```

```
Query OK, 1 row affected
```

```
mysql> SAVEPOINT sp1;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO `Transaction` VALUES (5, 1003, 2000,
'Debit');
```

```
Query OK, 1 row affected
```

```
mysql> ROLLBACK TO sp1;
```

```
Query OK, 0 rows affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Account;
```

```
+-----------+------------+-------------+----------+
| AccountNo | CustomerID | AccountType | Balance  |
+-----------+------------+-------------+----------+
|      1001 |          1 | Savings     | 45000.00 |
|      1002 |          2 | Current     | 70000.00 |
|      1003 |          3 | Savings     | 28000.00 |
|      1004 |          4 | Savings     | 15000.00 |
+-----------+------------+-------------+----------+
4 rows in set
```

```
mysql> SELECT * FROM `Transaction`;
```

```
+-------+-----------+----------+---------+
| TxnID | AccountNo | Amount   | TxnType |
+-------+-----------+----------+---------+
|     1 |      1001 |  5000.00 | Credit  |
|     2 |      1001 |  5000.00 | Debit   |
|     3 |      1002 | 10000.00 | Debit   |
|     4 |      1003 | 10000.00 | Credit  |
+-------+-----------+----------+---------+
4 rows in set
```

**Q6** **Delete a transaction record and use ROLLBACK to undo the deletion. Grant SELECT permission on Account to bank_clerk. Grant SELECT and INSERT permissions on Transaction to bank_clerk.**

**How to think**

- Restore recipe, then two GRANTs (backticks around Transaction).

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> DELETE FROM `Transaction` WHERE TxnID = 1;
```

```
Query OK, 1 row affected
```

```
mysql> ROLLBACK;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT COUNT(*) FROM `Transaction`;
```

```
+----------+
| COUNT(*) |
+----------+
|        4 |
+----------+
1 row in set
```

```
mysql> SET autocommit = 1;
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE USER IF NOT EXISTS 'bank_clerk'@'localhost'
IDENTIFIED BY 'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT SELECT ON bank.Account TO
'bank_clerk'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT SELECT, INSERT ON bank.`Transaction` TO
'bank_clerk'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> SHOW GRANTS FOR 'bank_clerk'@'localhost';
```

```
+-----------------------------------------------------------------------+
| Grants for bank_clerk@localhost                                       |
+-----------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `bank_clerk`@`localhost`                        |
| GRANT SELECT, INSERT ON `bank`.`Transaction` TO `bank_clerk`@`localhost` |
| GRANT SELECT ON `bank`.`Account` TO `bank_clerk`@`localhost`          |
+-----------------------------------------------------------------------+
3 rows in set
```

**Q7** Grant SELECT and UPDATE permissions on Account to bank_manager. Revoke UPDATE permission on Account

**from bank_clerk. Grant all privileges on Account to bank_admin.**

### How to think

- bank_clerk never got UPDATE above, so give it first, then revoke (otherwise MySQL says there is no such grant).

```
mysql> CREATE USER IF NOT EXISTS 'bank_manager'@'localhost'
IDENTIFIED BY 'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE USER IF NOT EXISTS 'bank_admin'@'localhost'
IDENTIFIED BY 'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT SELECT, UPDATE ON bank.Account TO
'bank_manager'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT UPDATE ON bank.Account TO
'bank_clerk'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> REVOKE UPDATE ON bank.Account FROM
'bank_clerk'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT ALL PRIVILEGES ON bank.Account TO
'bank_admin'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> SHOW GRANTS FOR 'bank_clerk'@'localhost';
```

```
+-----------------------------------------------------------------------+
| Grants for bank_clerk@localhost                                       |
+-----------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `bank_clerk`@`localhost`                        |
| GRANT SELECT, INSERT ON `bank`.`Transaction` TO `bank_clerk`@`localhost` |
| GRANT SELECT ON `bank`.`Account` TO `bank_clerk`@`localhost`          |
+-----------------------------------------------------------------------+
3 rows in set
```

**Q8** Create a view named Customer_Account_View to display CustomerID, CustomerName, AccountNo, AccountType, and Balance by joining Customer and Account. Create an index named idx_account_balance on the Balance column of the Account table.

### How to think

- 2-table JOIN on CustomerID + CREATE VIEW. Index on Account(Balance).

```
mysql> CREATE VIEW Customer_Account_View AS
    -> SELECT c.CustomerID, c.CustomerName, a.AccountNo,
a.AccountType, a.Balance
    -> FROM Customer c
    -> JOIN Account a ON c.CustomerID = a.CustomerID;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Customer_Account_View;
```

```
+------------+--------------+-----------+-------------+----------+
| CustomerID | CustomerName | AccountNo | AccountType | Balance  |
+------------+--------------+-----------+-------------+----------+
|          1 | Arun         |      1001 | Savings     | 45000.00 |
|          2 | Priya        |      1002 | Current     | 70000.00 |
|          3 | Kumar        |      1003 | Savings     | 28000.00 |
|          4 | Divya        |      1004 | Savings     | 15000.00 |
+------------+--------------+-----------+-------------+----------+
4 rows in set
```

```
mysql> CREATE INDEX idx_account_balance ON Account(Balance);
```

```
Query OK, 0 rows affected
```

**ALSO** Display the AccountNo, CustomerID, and Balance of accounts whose balance is greater than the average account balance. (ALL 4 papers)

**How to think**

- Average subquery.

```
mysql> SELECT AccountNo, CustomerID, Balance FROM Account
    -> WHERE Balance > (SELECT AVG(Balance) FROM Account);
```

```
+-----------+------------+----------+
| AccountNo | CustomerID | Balance  |
+-----------+------------+----------+
|      1001 |          1 | 45000.00 |
|      1002 |          2 | 70000.00 |
+-----------+------------+----------+
2 rows in set
```

# 6 · Airline Reservation

**Schema**

`Passenger(PassengerID, Name)`

`Flight(FlightID, FlightName, Source, Destination, TicketPrice)`

`Ticket(TicketID, PassengerID, FlightID, SeatNo)`

**Setup (not asked):** Create Passenger and Flight first.

```
mysql> CREATE DATABASE airline;
```

```
Query OK, 0 rows affected
```

```
mysql> USE airline;
```

```
Database changed
```

```
mysql> CREATE TABLE Passenger (
    ->     PassengerID INT PRIMARY KEY,
    ->     Name        VARCHAR(50) NOT NULL
    -> );
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE TABLE Flight (
    ->     FlightID    VARCHAR(10) PRIMARY KEY,
    ->     FlightName  VARCHAR(50),
    ->     Source      VARCHAR(30),
    ->     Destination VARCHAR(30),
    ->     TicketPrice DECIMAL(8,2)
    -> );
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Passenger VALUES (1, 'Arun'), (2,
'Priya');
```

```
Query OK, 2 rows affected
```

**Q1** **Insert two flights and modify the ticket price of Flight F1 by adding ₹500. Create Ticket with foreign keys to Passenger and Flight.**

**How to think**

- FlightID 'F1' has letters → VARCHAR. "adding ₹500" → SET TicketPrice = TicketPrice + 500.
- Ticket = middle table with two FKs (FlightID VARCHAR to match).

```
mysql> INSERT INTO Flight VALUES ('F1', 'IndiGo 6E201',
'Chennai', 'Delhi', 5500);
```

```
Query OK, 1 row affected
```

```
mysql> INSERT INTO Flight VALUES ('F2', 'Air India 440',
'Chennai', 'Mumbai', 4200);
```

```
Query OK, 1 row affected
```

```
mysql> UPDATE Flight SET TicketPrice = TicketPrice + 500
WHERE FlightID = 'F1';
```

```
Query OK, 1 row affected
```

```
mysql> SELECT * FROM Flight;
```

```
+----------+--------------+---------+-------------+-------------+
| FlightID | FlightName   | Source  | Destination | TicketPrice |
+----------+--------------+---------+-------------+-------------+
| F1       | IndiGo 6E201 | Chennai | Delhi       |     6000.00 |
| F2       | Air India 440 | Chennai | Mumbai      |     4200.00 |
+----------+--------------+---------+-------------+-------------+
2 rows in set
```

```
mysql> CREATE TABLE Ticket (
    ->     TicketID    INT PRIMARY KEY,
    ->     PassengerID INT,
    ->     FlightID    VARCHAR(10),
    ->     SeatNo      VARCHAR(5),
    ->     FOREIGN KEY (PassengerID) REFERENCES
Passenger(PassengerID),
    ->     FOREIGN KEY (FlightID)    REFERENCES
```

```
Flight(FlightID)
    -> );
```

```
Query OK, 0 rows affected
```

## Q2 Display flights ordered by ticket price in descending order. Display all possible passenger–flight combinations.

### How to think

- ORDER BY TicketPrice DESC.
- "all possible combinations" → CROSS JOIN (rows = passengers × flights).

```
mysql> SELECT * FROM Flight ORDER BY TicketPrice DESC;
```

```
+----------+--------------+---------+-------------+-------------+
| FlightID | FlightName   | Source  | Destination | TicketPrice |
+----------+--------------+---------+-------------+-------------+
| F1       | IndiGo 6E201 | Chennai | Delhi       |     6000.00 |
| F2       | Air India 440 | Chennai | Mumbai     |     4200.00 |
+----------+--------------+---------+-------------+-------------+
2 rows in set
```

```
mysql> SELECT p.Name, f.FlightName
    -> FROM Passenger p CROSS JOIN Flight f;
```

```
+-------+---------------+
| Name  | FlightName    |
+-------+---------------+
| Arun  | IndiGo 6E201  |
| Priya | IndiGo 6E201  |
| Arun  | Air India 440 |
| Priya | Air India 440 |
+-------+---------------+
4 rows in set
```

## Q3 Insert a new passenger and book a ticket. Use COMMIT.

### How to think

- COMMIT recipe: passenger then ticket.

```
mysql> SET autocommit = 0;
```

```
Query OK, 0 rows affected
```

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Passenger VALUES (3, 'Kumar');
```

```
Query OK, 1 row affected
```

```
mysql> INSERT INTO Ticket VALUES (1, 3, 'F1', '12A');
```

```
Query OK, 1 row affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Ticket;
```

```
+----------+-------------+----------+--------+
| TicketID | PassengerID | FlightID | SeatNo |
+----------+-------------+----------+--------+
|        1 |           3 | F1       | 12A    |
+----------+-------------+----------+--------+
1 row in set
```

**Q4** **Update the ticket's seat number and use ROLLBACK to cancel the change.**

**How to think**

- ROLLBACK recipe; SELECT shows the seat is still 12A.

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> UPDATE Ticket SET SeatNo = '20C' WHERE TicketID = 1;
```

```
Query OK, 1 row affected
```

```
mysql> SELECT * FROM Ticket;
```

```
+----------+-------------+----------+--------+
| TicketID | PassengerID | FlightID | SeatNo |
+----------+-------------+----------+--------+
|        1 |           3 | F1       | 20C    |
+----------+-------------+----------+--------+
1 row in set
```

```
mysql> ROLLBACK;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Ticket;
```

```
+----------+-------------+----------+--------+
| TicketID | PassengerID | FlightID | SeatNo |
+----------+-------------+----------+--------+
|        1 |           3 | F1       | 12A    |
+----------+-------------+----------+--------+
1 row in set
```

## Q5 Insert a passenger, create a savepoint, book a ticket, and roll back to the savepoint.

### How to think

- A = passenger (kept), B = ticket (undone).

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Passenger VALUES (4, 'Divya');
```

```
Query OK, 1 row affected
```

```
mysql> SAVEPOINT sp1;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Ticket VALUES (2, 4, 'F2', '5B');
```

```
Query OK, 1 row affected
```

```
mysql> ROLLBACK TO sp1;
```

```
Query OK, 0 rows affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Passenger WHERE PassengerID = 4;
```

```
+-------------+-------+
| PassengerID | Name  |
+-------------+-------+
|           4 | Divya |
+-------------+-------+
1 row in set
```

```
mysql> SELECT * FROM Ticket WHERE PassengerID = 4;
```

```
Empty set
```

## Q6 Delete a ticket record and use ROLLBACK to restore it.

### How to think

- Restore recipe.

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> DELETE FROM Ticket WHERE TicketID = 1;
```

```
Query OK, 1 row affected
```

```
mysql> SELECT * FROM Ticket;
```

```
Empty set
```

```
mysql> ROLLBACK;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Ticket;
```

```
+----------+-------------+----------+--------+
| TicketID | PassengerID | FlightID | SeatNo |
+----------+-------------+----------+--------+
|        1 |           3 | F1       | 12A    |
+----------+-------------+----------+--------+
1 row in set
```

**Q7** **Book two tickets, create a savepoint after the first booking, and roll back the second booking using the savepoint.**

**How to think**

- Ticket 1 → SAVEPOINT → ticket 2 → ROLLBACK TO → COMMIT. Only the first stays.

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Ticket VALUES (3, 1, 'F1', '1A');
```

```
Query OK, 1 row affected
```

```
mysql> SAVEPOINT after_first;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Ticket VALUES (4, 2, 'F2', '2B');
```

```
Query OK, 1 row affected
```

```
mysql> ROLLBACK TO after_first;
```

```
Query OK, 0 rows affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Ticket;
```

```
+----------+-------------+----------+--------+
| TicketID | PassengerID | FlightID | SeatNo |
+----------+-------------+----------+--------+
|        1 |           3 | F1       | 12A    |
|        3 |           1 | F1       | 1A     |
+----------+-------------+----------+--------+
2 rows in set
```

**Q8** Grant SELECT permission on Flight to flight_user. Grant SELECT and INSERT permissions on Ticket to booking_user. Grant SELECT, INSERT, and UPDATE permissions on Passenger to reservation_user. Revoke INSERT permission on Ticket from booking_user. Grant all privileges on Ticket to airline_admin.

**How to think**

- 5 sentences → create 4 users, then 5 lines (one is REVOKE ... FROM).

```
mysql> SET autocommit = 1;
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE USER IF NOT EXISTS 'flight_user'@'localhost'
IDENTIFIED BY 'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE USER IF NOT EXISTS 'booking_user'@'localhost'
IDENTIFIED BY 'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE USER IF NOT EXISTS
'reservation_user'@'localhost' IDENTIFIED BY 'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE USER IF NOT EXISTS 'airline_admin'@'localhost'
IDENTIFIED BY 'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT SELECT ON airline.Flight TO
'flight_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT SELECT, INSERT ON airline.Ticket TO
'booking_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT SELECT, INSERT, UPDATE ON airline.Passenger TO
'reservation_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> REVOKE INSERT ON airline.Ticket FROM
'booking_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT ALL PRIVILEGES ON airline.Ticket TO
'airline_admin'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> SHOW GRANTS FOR 'booking_user'@'localhost';
```

```
+-------------------------------------------------------------------+
| Grants for booking_user@localhost                                 |
+-------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `booking_user`@`localhost`                  |
| GRANT SELECT ON `airline`.`Ticket` TO `booking_user`@`localhost`  |
| GRANT SELECT ON `railway`.`Booking` TO `booking_user`@`localhost` |
+-------------------------------------------------------------------+
3 rows in set
```

**Q9** Create a view named Ticket_Details_View to display TicketID, passenger Name, FlightName, Source, Destination, and SeatNo by joining Passenger, Flight, and

## Ticket. Create an index named idx_flight_source on the Source column of the Flight table.

### How to think

- Middle table Ticket + two JOINs. Index on Flight(Source).

```
mysql> CREATE VIEW Ticket_Details_View AS
    -> SELECT t.TicketID, p.Name, f.FlightName, f.Source,
f.Destination, t.SeatNo
    -> FROM Ticket t
    -> JOIN Passenger p ON t.PassengerID = p.PassengerID
    -> JOIN Flight f    ON t.FlightID    = f.FlightID;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Ticket_Details_View;
```

```
+----------+-------+--------------+---------+-------------+--------+
| TicketID | Name  | FlightName   | Source  | Destination | SeatNo |
+----------+-------+--------------+---------+-------------+--------+
|        1 | Kumar | IndiGo 6E201 | Chennai | Delhi       | 12A    |
|        3 | Arun  | IndiGo 6E201 | Chennai | Delhi       | 1A     |
+----------+-------+--------------+---------+-------------+--------+
2 rows in set
```

```
mysql> CREATE INDEX idx_flight_source ON Flight(Source);
```

```
Query OK, 0 rows affected
```

# 7 · Movie Booking

**Schema**

```
Movie(MovieID, MovieName, Genre, TicketPrice)
Customer(CustomerID, CustomerName)
Booking(BookingID, CustomerID, MovieID, SeatsBooked)
```

## Q1 Create the Movie table with TicketPrice DEFAULT 150 and insert movies without specifying the ticket price for one row.

**How to think**

- DEFAULT 150 on TicketPrice.
- To skip the price, INSERT with a column list that leaves TicketPrice out → it becomes 150.

```
mysql> CREATE DATABASE cinema;
```

```
Query OK, 0 rows affected
```

```
mysql> USE cinema;
```

```
Database changed
```

```
mysql> CREATE TABLE Movie (
    ->     MovieID     INT PRIMARY KEY,
    ->     MovieName   VARCHAR(50) NOT NULL,
    ->     Genre       VARCHAR(20),
    ->     TicketPrice DECIMAL(6,2) DEFAULT 150
    -> );
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Movie VALUES (1, 'Leo', 'Action', 200);
```

```
Query OK, 1 row affected
```

```
mysql> INSERT INTO Movie (MovieID, MovieName, Genre) VALUES
(2, 'Jailer', 'Action');
```

```
Query OK, 1 row affected
```

```
mysql> INSERT INTO Movie VALUES (3, 'Vikram', 'Thriller',
180), (4, 'Amaran', 'Drama', 170);
```

```
Query OK, 2 rows affected
```

```
mysql> SELECT * FROM Movie;
```

```
+---------+-----------+----------+-------------+
| MovieID | MovieName | Genre    | TicketPrice |
+---------+-----------+----------+-------------+
|       1 | Leo       | Action   |      200.00 |
|       2 | Jailer    | Action   |      150.00 |
|       3 | Vikram    | Thriller |      180.00 |
|       4 | Amaran    | Drama    |      170.00 |
+---------+-----------+----------+-------------+
4 rows in set
```

**Setup (not asked):** Customer and Booking tables with some bookings.

```
mysql> CREATE TABLE Customer (
    ->     CustomerID   INT PRIMARY KEY,
    ->     CustomerName VARCHAR(50) NOT NULL
    -> );
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE TABLE Booking (
    ->     BookingID   INT PRIMARY KEY,
    ->     CustomerID  INT,
    ->     MovieID     INT,
    ->     SeatsBooked INT,
    ->     FOREIGN KEY (CustomerID) REFERENCES
Customer(CustomerID),
    ->     FOREIGN KEY (MovieID)    REFERENCES Movie(MovieID)
    -> );
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Customer VALUES (1, 'Arun'), (2, 'Priya');
```

```
Query OK, 2 rows affected
```

```
mysql> INSERT INTO Booking VALUES (1, 1, 1, 4), (2, 2, 1, 5),
(3, 1, 2, 2), (4, 2, 3, 3);
```

```
Query OK, 4 rows affected
```

## Q2 Find the average seats booked for each movie and show only movies with average seats greater than 3.

### How to think

- "for each movie" → GROUP BY MovieID. Condition on AVG → HAVING.

```
mysql> SELECT MovieID, AVG(SeatsBooked) AS AvgSeats
    -> FROM Booking
    -> GROUP BY MovieID
    -> HAVING AVG(SeatsBooked) > 3;
```

```
+---------+----------+
| MovieID | AvgSeats |
+---------+----------+
|       1 |   4.5000 |
+---------+----------+
1 row in set
```

## Q3 Show every booking and every movie, matched where possible, with blanks for unmatched rows.

### How to think

- That wording = FULL OUTER JOIN. MySQL has no keyword → LEFT JOIN UNION RIGHT JOIN. Amaran (no booking) shows NULLs.

```
mysql> SELECT b.BookingID, b.SeatsBooked, m.MovieID,
m.MovieName
    -> FROM Booking b LEFT JOIN Movie m ON b.MovieID =
m.MovieID
```

```
    -> UNION
    -> SELECT b.BookingID, b.SeatsBooked, m.MovieID,
m.MovieName
    -> FROM Booking b RIGHT JOIN Movie m ON b.MovieID =
m.MovieID;
```

```
+-----------+-------------+---------+-----------+
| BookingID | SeatsBooked | MovieID | MovieName |
+-----------+-------------+---------+-----------+
|         1 |           4 |       1 | Leo       |
|         2 |           5 |       1 | Leo       |
|         3 |           2 |       2 | Jailer    |
|         4 |           3 |       3 | Vikram    |
| NULL      | NULL        |       4 | Amaran    |
+-----------+-------------+---------+-----------+
5 rows in set
```

## Q4 Insert a new customer and book tickets for a movie. Use COMMIT.

### How to think

- COMMIT recipe: customer then booking.

```
mysql> SET autocommit = 0;
```

```
Query OK, 0 rows affected
```

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Customer VALUES (3, 'Kumar');
```

```
Query OK, 1 row affected
```

```
mysql> INSERT INTO Booking VALUES (5, 3, 3, 3);
```

```
Query OK, 1 row affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Booking WHERE CustomerID = 3;
```

```
+-----------+------------+---------+-------------+
| BookingID | CustomerID | MovieID | SeatsBooked |
+-----------+------------+---------+-------------+
|         5 |          3 |       3 |           3 |
+-----------+------------+---------+-------------+
1 row in set
```

## Q5 Update the number of seats booked and use ROLLBACK.

**How to think**

- ROLLBACK recipe.

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> UPDATE Booking SET SeatsBooked = 10 WHERE BookingID =
5;
```

```
Query OK, 1 row affected
```

```
mysql> ROLLBACK;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Booking WHERE BookingID = 5;
```

```
+-----------+------------+---------+-------------+
| BookingID | CustomerID | MovieID | SeatsBooked |
+-----------+------------+---------+-------------+
|         5 |          3 |       3 |           3 |
+-----------+------------+---------+-------------+
1 row in set
```

## Q6 Create a booking, create a savepoint, modify the number of seats, and roll back to the savepoint.

**How to think**

- A = new booking (kept), B = seat change (undone).

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Booking VALUES (6, 2, 3, 2);
```

```
Query OK, 1 row affected
```

```
mysql> SAVEPOINT sp1;
```

```
Query OK, 0 rows affected
```

```
mysql> UPDATE Booking SET SeatsBooked = 6 WHERE BookingID =
6;
```

```
Query OK, 1 row affected
```

```
mysql> ROLLBACK TO sp1;
```

```
Query OK, 0 rows affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Booking WHERE BookingID = 6;
```

```
+-----------+------------+---------+-------------+
| BookingID | CustomerID | MovieID | SeatsBooked |
+-----------+------------+---------+-------------+
|         6 |          2 |       3 |           2 |
+-----------+------------+---------+-------------+
1 row in set
```

## Q7 Delete a booking and use ROLLBACK to restore it.

**How to think**

- Restore recipe.

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> DELETE FROM Booking WHERE BookingID = 6;
```

```
Query OK, 1 row affected
```

```
mysql> ROLLBACK;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Booking WHERE BookingID = 6;
```

```
+-----------+------------+---------+-------------+
| BookingID | CustomerID | MovieID | SeatsBooked |
+-----------+------------+---------+-------------+
|         6 |          2 |       3 |           2 |
+-----------+------------+---------+-------------+
1 row in set
```

## Q8 Insert two bookings, create a savepoint between them, and roll back only the second booking.

**How to think**

- Booking 1 → SAVEPOINT → booking 2 → ROLLBACK TO → COMMIT.

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Booking VALUES (7, 1, 3, 1);
```

```
Query OK, 1 row affected
```

```
mysql> SAVEPOINT between_bookings;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Booking VALUES (8, 2, 2, 2);
```

```
Query OK, 1 row affected
```

```
mysql> ROLLBACK TO between_bookings;
```

```
Query OK, 0 rows affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Booking WHERE BookingID IN (7, 8);
```

```
+-----------+------------+---------+-------------+
| BookingID | CustomerID | MovieID | SeatsBooked |
+-----------+------------+---------+-------------+
|         7 |          1 |       3 |           1 |
+-----------+------------+---------+-------------+
1 row in set
```

**Q9** Grant SELECT permission on Movie to movie_user. Grant SELECT and INSERT permissions on Booking to booking_user. Grant SELECT, INSERT, and UPDATE permissions on Customer to customer_user. Revoke INSERT permission on Booking from booking_user. Grant all privileges on Booking to movie_admin.

**How to think**

- Create 4 users, then 5 lines.

```
mysql> SET autocommit = 1;
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE USER IF NOT EXISTS 'movie_user'@'localhost'
IDENTIFIED BY 'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE USER IF NOT EXISTS 'booking_user'@'localhost'
IDENTIFIED BY 'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE USER IF NOT EXISTS 'customer_user'@'localhost'
IDENTIFIED BY 'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE USER IF NOT EXISTS 'movie_admin'@'localhost'
IDENTIFIED BY 'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT SELECT ON cinema.Movie TO
'movie_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT SELECT, INSERT ON cinema.Booking TO
'booking_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT SELECT, INSERT, UPDATE ON cinema.Customer TO
'customer_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> REVOKE INSERT ON cinema.Booking FROM
'booking_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT ALL PRIVILEGES ON cinema.Booking TO
'movie_admin'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> SHOW GRANTS FOR 'booking_user'@'localhost';
```

```
+---------------------------------------------------------------------+
| Grants for booking_user@localhost                                   |
+---------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `booking_user`@`localhost`                    |
| GRANT SELECT ON `cinema`.`Booking` TO `booking_user`@`localhost`    |
| GRANT SELECT ON `railway`.`Booking` TO `booking_user`@`localhost`   |
| GRANT SELECT ON `airline`.`Ticket` TO `booking_user`@`localhost`    |
+---------------------------------------------------------------------+
4 rows in set
```

**Q10** **Create a view named Movie_Booking_View to display BookingID, CustomerName, MovieName, Genre, and SeatsBooked by joining Customer, Movie, and Booking. Create an index named idx_movie_genre on the Genre column of the Movie table.**

### How to think

- Middle table Booking + two JOINs. Index on Movie(Genre).

```
mysql> CREATE VIEW Movie_Booking_View AS
    -> SELECT b.BookingID, c.CustomerName, m.MovieName,
m.Genre, b.SeatsBooked
    -> FROM Booking b
    -> JOIN Customer c ON b.CustomerID = c.CustomerID
    -> JOIN Movie m    ON b.MovieID    = m.MovieID;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Movie_Booking_View;
```

```
+-----------+--------------+-----------+----------+-------------+
| BookingID | CustomerName | MovieName | Genre    | SeatsBooked |
+-----------+--------------+-----------+----------+-------------+
|         1 | Arun         | Leo       | Action   |           4 |
|         2 | Priya        | Leo       | Action   |           5 |
|         3 | Arun         | Jailer    | Action   |           2 |
|         4 | Priya        | Vikram    | Thriller |           3 |
|         5 | Kumar        | Vikram    | Thriller |           3 |
|         6 | Priya        | Vikram    | Thriller |           2 |
|         7 | Arun         | Vikram    | Thriller |           1 |
+-----------+--------------+-----------+----------+-------------+
7 rows in set
```

```
mysql> CREATE INDEX idx_movie_genre ON Movie(Genre);
```

```
Query OK, 0 rows affected
```

# 8 · Food Delivery

**Schema**

```
Restaurant(RestaurantID, RestaurantName, City)
Customer(CustomerID, CustomerName, City)
FoodOrder(OrderID, CustomerID, RestaurantID, Quantity, Price, Rating)
```

**Setup (not asked):** Parent tables first.

```
mysql> CREATE DATABASE food;
```

```
Query OK, 0 rows affected
```

```
mysql> USE food;
```

```
Database changed
```

```
mysql> CREATE TABLE Restaurant (
    ->     RestaurantID   INT PRIMARY KEY,
    ->     RestaurantName VARCHAR(50) NOT NULL,
    ->     City           VARCHAR(30)
    -> );
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE TABLE Customer (
    ->     CustomerID   INT PRIMARY KEY,
    ->     CustomerName VARCHAR(50) NOT NULL,
    ->     City         VARCHAR(30)
    -> );
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Restaurant VALUES (1, 'A2B', 'Chennai'),
(2, 'Murugan Idli', 'Madurai'),
    ->                                (3, 'Dindigul Biryani',
'Chennai'), (4, 'Adyar Bakery', 'Chennai');
```

```
Query OK, 4 rows affected
```

```
mysql> INSERT INTO Customer VALUES (1, 'Arun', 'Chennai'),
(2, 'Priya', 'Madurai');
```

```
Query OK, 2 rows affected
```

## Q1 Create FoodOrder with foreign keys to Customer and Restaurant and use ON UPDATE CASCADE.

### How to think

- Two FOREIGN KEY lines, each ending ON UPDATE CASCADE.
- Explain: if a RestaurantID changes in Restaurant, it changes automatically in FoodOrder. Prove it with an UPDATE.

```
mysql> CREATE TABLE FoodOrder (
    ->     OrderID      INT PRIMARY KEY,
    ->     CustomerID   INT,
    ->     RestaurantID INT,
    ->     Quantity     INT,
    ->     Price        DECIMAL(8,2),
    ->     Rating       DECIMAL(2,1),
    ->     FOREIGN KEY (CustomerID)   REFERENCES
Customer(CustomerID)     ON UPDATE CASCADE,
    ->     FOREIGN KEY (RestaurantID) REFERENCES
Restaurant(RestaurantID) ON UPDATE CASCADE
    -> );
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO FoodOrder VALUES (1, 1, 1, 2, 300, 4.5),
(2, 2, 2, 3, 150, 5.0),
    ->                               (3, 1, 1, 1, 200, 3.9),
(4, 2, 3, 2, 400, 4.8);
```

```
Query OK, 4 rows affected
```

```
mysql> UPDATE Restaurant SET RestaurantID = 10 WHERE
RestaurantID = 1;
```

```
Query OK, 1 row affected
```

```
mysql> SELECT * FROM FoodOrder;
```

```
+---------+------------+--------------+----------+--------+--------+
| OrderID | CustomerID | RestaurantID | Quantity | Price  | Rating |
+---------+------------+--------------+----------+--------+--------+
|       1 |          1 |           10 |        2 | 300.00 |    4.5 |
|       2 |          2 |            2 |        3 | 150.00 |    5.0 |
|       3 |          1 |           10 |        1 | 200.00 |    3.9 |
|       4 |          2 |            3 |        2 | 400.00 |    4.8 |
+---------+------------+--------------+----------+--------+--------+
4 rows in set
```

## Q2 For each city, display total orders, average price, and total quantity where rating is at least 4.2.

### How to think

- "for each city" → GROUP BY City (City is in Restaurant → JOIN).
- "rating at least 4.2" is a row rule → WHERE Rating >= 4.2 (before grouping).
- total orders = COUNT, average price = AVG, total quantity = SUM.

```
mysql> SELECT r.City, COUNT(f.OrderID) AS TotalOrders,
    ->        AVG(f.Price) AS AvgPrice, SUM(f.Quantity) AS
TotalQty
    -> FROM FoodOrder f
    -> JOIN Restaurant r ON f.RestaurantID = r.RestaurantID
    -> WHERE f.Rating >= 4.2
    -> GROUP BY r.City;
```

```
+---------+-------------+------------+----------+
| City    | TotalOrders | AvgPrice   | TotalQty |
+---------+-------------+------------+----------+
| Chennai |           2 | 350.000000 |        4 |
| Madurai |           1 | 150.000000 |        3 |
+---------+-------------+------------+----------+
2 rows in set
```

**Q3** Find restaurants that have received at least one order with rating 5, and also write a query to find restaurants that have never received any order.

### How to think

- "at least one ... rating 5" → IN (SELECT ... WHERE Rating = 5).
- "never" → NOT IN.

```
mysql> SELECT * FROM Restaurant
    -> WHERE RestaurantID IN (SELECT RestaurantID FROM
FoodOrder WHERE Rating = 5);
```

```
+--------------+----------------+---------+
| RestaurantID | RestaurantName | City    |
+--------------+----------------+---------+
|            2 | Murugan Idli   | Madurai |
+--------------+----------------+---------+
1 row in set
```

```
mysql> SELECT * FROM Restaurant
    -> WHERE RestaurantID NOT IN (SELECT RestaurantID FROM
FoodOrder);
```

```
+--------------+----------------+---------+
| RestaurantID | RestaurantName | City    |
+--------------+----------------+---------+
|            4 | Adyar Bakery   | Chennai |
+--------------+----------------+---------+
1 row in set
```

**Q4** Insert a new customer and place a food order. Use COMMIT.

### How to think

- COMMIT recipe.

```
mysql> SET autocommit = 0;
```

```
Query OK, 0 rows affected
```

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Customer VALUES (3, 'Kumar', 'Salem');
```

```
Query OK, 1 row affected
```

```
mysql> INSERT INTO FoodOrder VALUES (5, 3, 2, 2, 180, 4.0);
```

```
Query OK, 1 row affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM FoodOrder WHERE OrderID = 5;
```

```
+---------+------------+--------------+----------+--------+--------+
| OrderID | CustomerID | RestaurantID | Quantity | Price  | Rating |
+---------+------------+--------------+----------+--------+--------+
|       5 |          3 |            2 |        2 | 180.00 |    4.0 |
+---------+------------+--------------+----------+--------+--------+
1 row in set
```

## Q5 Update the quantity and price of an order and use ROLLBACK.

**How to think**

- ROLLBACK recipe on SET Quantity = ..., Price = ....

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> UPDATE FoodOrder SET Quantity = 5, Price = 450 WHERE
OrderID = 5;
```

```
Query OK, 1 row affected
```

```
mysql> ROLLBACK;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM FoodOrder WHERE OrderID = 5;
```

```
+---------+------------+--------------+----------+--------+--------+
| OrderID | CustomerID | RestaurantID | Quantity | Price  | Rating |
+---------+------------+--------------+----------+--------+--------+
|       5 |          3 |            2 |        2 | 180.00 |    4.0 |
+---------+------------+--------------+----------+--------+--------+
1 row in set
```

## Q6 Place an order, create a savepoint, update its rating, and roll back to the savepoint.

### How to think

- A = new order (kept), B = rating change (undone).

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO FoodOrder VALUES (6, 2, 3, 1, 250, 4.0);
```

```
Query OK, 1 row affected
```

```
mysql> SAVEPOINT sp1;
```

```
Query OK, 0 rows affected
```

```
mysql> UPDATE FoodOrder SET Rating = 2.0 WHERE OrderID = 6;
```

```
Query OK, 1 row affected
```

```
mysql> ROLLBACK TO sp1;
```

```
Query OK, 0 rows affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM FoodOrder WHERE OrderID = 6;
+---------+------------+--------------+----------+--------+--------+
| OrderID | CustomerID | RestaurantID | Quantity | Price  | Rating |
+---------+------------+--------------+----------+--------+--------+
|       6 |          2 |            3 |        1 | 250.00 |    4.0 |
+---------+------------+--------------+----------+--------+--------+
1 row in set
```

## Q7 Delete a food order and use ROLLBACK to restore it.

### How to think

- Restore recipe.

```
mysql> START TRANSACTION;
Query OK, 0 rows affected
```

```
mysql> DELETE FROM FoodOrder WHERE OrderID = 6;
Query OK, 1 row affected
```

```
mysql> ROLLBACK;
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM FoodOrder WHERE OrderID = 6;
+---------+------------+--------------+----------+--------+--------+
| OrderID | CustomerID | RestaurantID | Quantity | Price  | Rating |
+---------+------------+--------------+----------+--------+--------+
|       6 |          2 |            3 |        1 | 250.00 |    4.0 |
+---------+------------+--------------+----------+--------+--------+
1 row in set
```

## Q8 Insert a new customer, place an order, create a savepoint, place another order, and roll back the second order.

### How to think

- Customer + order 1 (kept) → SAVEPOINT → order 2 → ROLLBACK TO.

```
mysql> START TRANSACTION;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO Customer VALUES (4, 'Divya', 'Trichy');
```

```
Query OK, 1 row affected
```

```
mysql> INSERT INTO FoodOrder VALUES (7, 4, 2, 2, 160, 4.8);
```

```
Query OK, 1 row affected
```

```
mysql> SAVEPOINT sp2;
```

```
Query OK, 0 rows affected
```

```
mysql> INSERT INTO FoodOrder VALUES (8, 4, 3, 1, 220, 4.1);
```

```
Query OK, 1 row affected
```

```
mysql> ROLLBACK TO sp2;
```

```
Query OK, 0 rows affected
```

```
mysql> COMMIT;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM FoodOrder WHERE CustomerID = 4;
```

```
+---------+------------+--------------+----------+--------+--------+
| OrderID | CustomerID | RestaurantID | Quantity | Price  | Rating |
+---------+------------+--------------+----------+--------+--------+
|       7 |          4 |            2 |        2 | 160.00 |    4.8 |
+---------+------------+--------------+----------+--------+--------+
1 row in set
```

**Q9** Grant SELECT permission on Restaurant to delivery_user. Grant SELECT and INSERT permissions on FoodOrder to order_user. Grant SELECT, INSERT, and UPDATE permissions on Customer to customer_user. Revoke INSERT permission on FoodOrder from

**order_user. Grant all privileges on FoodOrder to restaurant_admin.**

**How to think**

- Create 4 users, then 5 lines.

```
mysql> SET autocommit = 1;
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE USER IF NOT EXISTS 'delivery_user'@'localhost'
IDENTIFIED BY 'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE USER IF NOT EXISTS 'order_user'@'localhost'
IDENTIFIED BY 'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE USER IF NOT EXISTS 'customer_user'@'localhost'
IDENTIFIED BY 'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> CREATE USER IF NOT EXISTS
'restaurant_admin'@'localhost' IDENTIFIED BY 'pass123';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT SELECT ON food.Restaurant TO
'delivery_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT SELECT, INSERT ON food.FoodOrder TO
'order_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT SELECT, INSERT, UPDATE ON food.Customer TO
'customer_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> REVOKE INSERT ON food.FoodOrder FROM
'order_user'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> GRANT ALL PRIVILEGES ON food.FoodOrder TO
'restaurant_admin'@'localhost';
```

```
Query OK, 0 rows affected
```

```
mysql> SHOW GRANTS FOR 'order_user'@'localhost';
```

```
+-------------------------------------------------------------------+
| Grants for order_user@localhost                                   |
+-------------------------------------------------------------------+
| GRANT USAGE ON *.* TO `order_user`@`localhost`                    |
| GRANT SELECT ON `food`.`FoodOrder` TO `order_user`@`localhost` |
+-------------------------------------------------------------------+
2 rows in set
```

---

**ALSO** Create a view named Food_Order_View to display OrderID, CustomerName, RestaurantName, Quantity, Price, and Rating by joining Customer, Restaurant, and FoodOrder. (Set 1)

**How to think**

- Middle table FoodOrder + two JOINs.

```
mysql> CREATE VIEW Food_Order_View AS
    -> SELECT f.OrderID, c.CustomerName, r.RestaurantName,
f.Quantity, f.Price, f.Rating
    -> FROM FoodOrder f
    -> JOIN Customer c   ON f.CustomerID   = c.CustomerID
    -> JOIN Restaurant r ON f.RestaurantID = r.RestaurantID;
```

```
Query OK, 0 rows affected
```

```
mysql> SELECT * FROM Food_Order_View;
```

```
+---------+--------------+-----------------+----------+--------+--------+
| OrderID | CustomerName | RestaurantName  | Quantity | Price  | Rating |
+---------+--------------+-----------------+----------+--------+--------+
|       1 | Arun         | A2B             |        2 | 300.00 |    4.5 |
|       2 | Priya        | Murugan Idli    |        3 | 150.00 |    5.0 |
|       3 | Arun         | A2B             |        1 | 200.00 |    3.9 |
|       4 | Priya        | Dindigul Biryani |        2 | 400.00 |    4.8 |
|       5 | Kumar        | Murugan Idli    |        2 | 180.00 |    4.0 |
|       6 | Priya        | Dindigul Biryani |        1 | 250.00 |    4.0 |
|       7 | Divya        | Murugan Idli    |        2 | 160.00 |    4.8 |
+---------+--------------+-----------------+----------+--------+--------+
7 rows in set
```